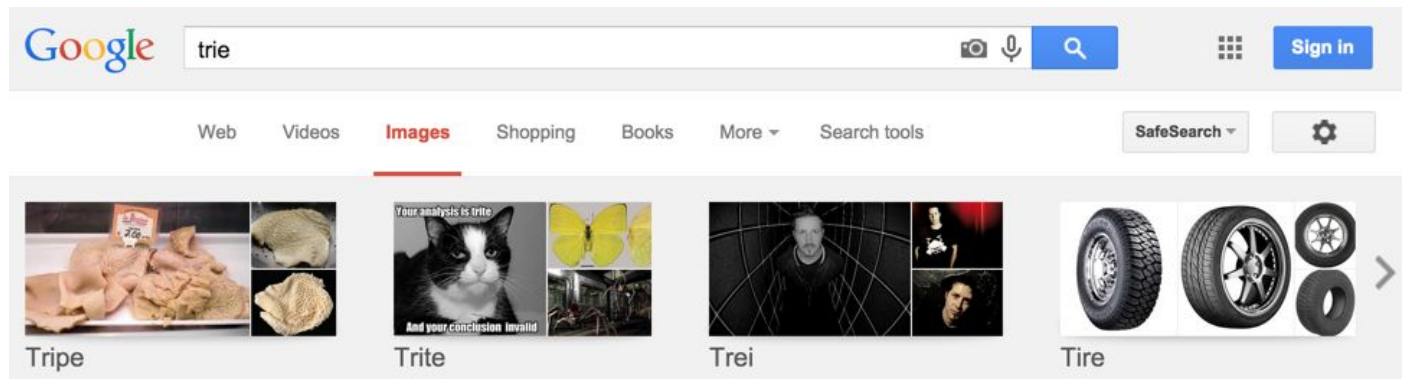




Lecture 29

Tries

CS61B, Fall 2025 @ UC Berkeley

Slides credit: Josh Hug

Tries (Conceptual)

Lecture 29, CS61B, Fall 2025

Tries (Conceptual)

Trie Implementation and
Performance

Alternate Child Tracking Strategies

Trie String Operations

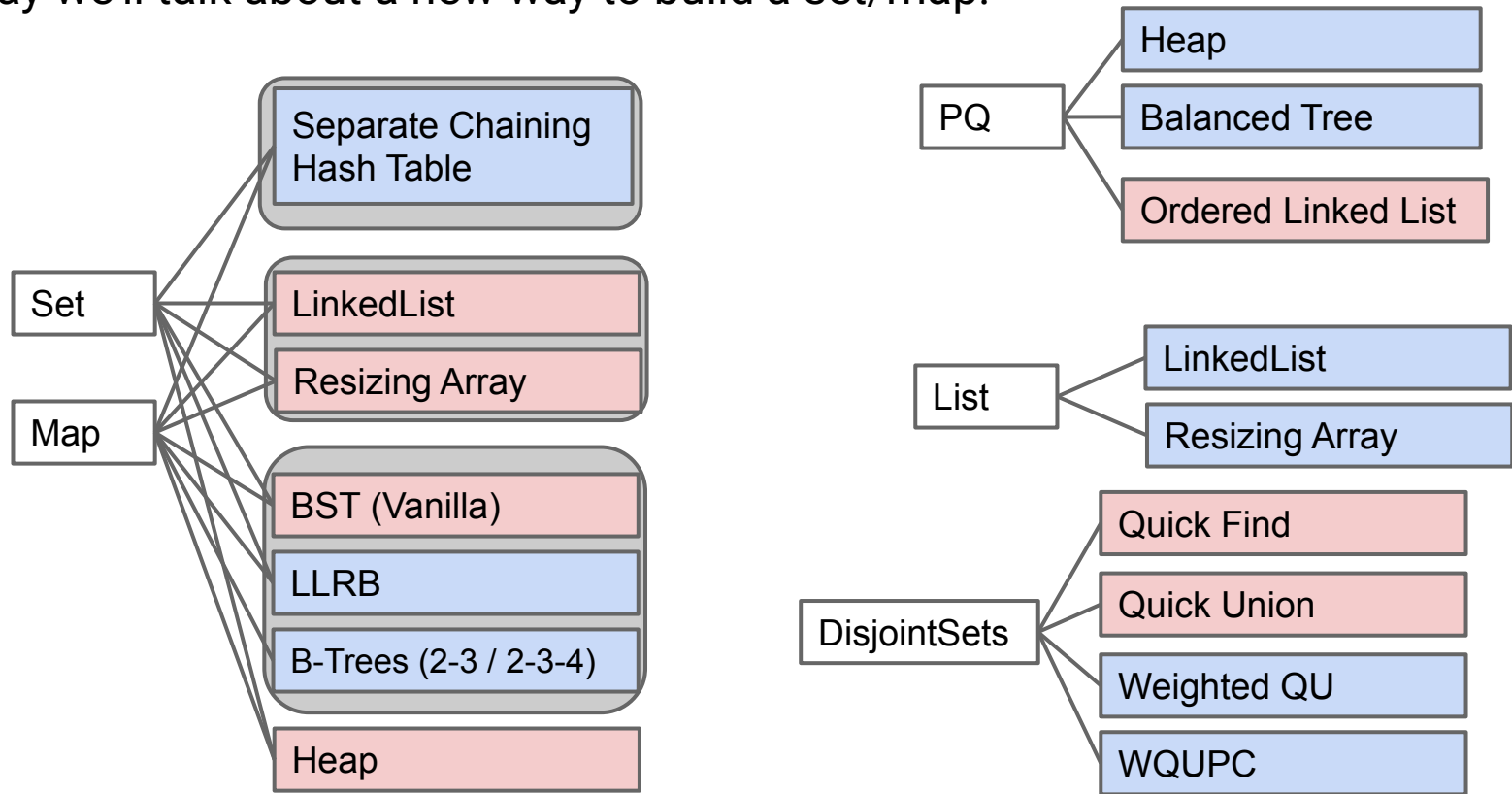
Autocomplete

Trie Summary

Abstract Data Types vs. Specific Implementations

There are many ways to implement an abstract data type.

- Today we'll talk about a new way to build a set/map.



BST and Hash Table Set Runtimes

Runtimes for our Balanced Search Tree and Hash Table implementations were very fast.

If we know that our keys all have some common special property, we can sometimes get even better implementations.

Example: Suppose we know our keys are always single ASCII characters.

- e.g. 'a', 'g', '!'

	contains(x)	add(x)
Balanced BST	$\Theta(\log N)$	$\Theta(\log N)$
Resizing Separate Chaining Hash Table	$\Theta(1)$ assuming even spread	$\Theta(1)$ on average, assuming even spread

Special Case 1: Character Keyed Map

Suppose we know that our keys are always ASCII characters.

- Can just use an array!
- Simple and fast.

```
public class DataIndexedCharMap<V> {  
    private V[] items;  
    public DataIndexedCharMap(int R) {  
        items = (V[]) new Object[R];  
    }  
    public void put(char c, V val) {  
        items[c] = val;  
    }  
    public V get(char c) {  
        return items[c];  
    }  
}
```

	key type	get(x)	add(x)
Balanced BST	comparable	$\Theta(\log N)$	$\Theta(\log N)$
Resizing Separate Chaining Hash Table	hashable	$\Theta(1)$ assuming even spread	$\Theta(1)$ on average, assuming even spread
data indexed array	chars	$\Theta(1)$	$\Theta(1)$

R is the number of possible characters, e.g. 128 for ASCII.

Special Case 2: String Keyed Map

Suppose we know that our keys are always strings.

- Can use a special data structure known as a Trie.
- Basic idea: Store each letter of the string as a node in a tree.

Tries will have great performance on:

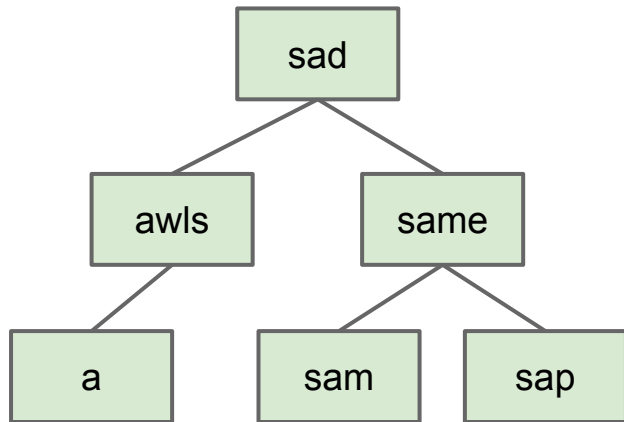
- get
- add
- special string operations

	key type	get(x)	add(x)
Balanced BST	comparable	$\Theta(\log N)$	$\Theta(\log N)$
Resizing Separate Chaining Hash Table	hashable	$\Theta(1)$ assuming even spread	$\Theta(1)$ on average, assuming even spread
data indexed array	chars	$\Theta(1)$	$\Theta(1)$
Tries	Strings	?	?

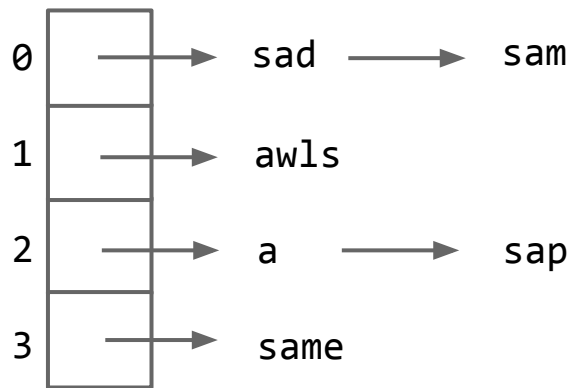
Sets of Strings

Suppose we have a set containing “sam”, “sad”, “sap”, “same”, “a”, and “awls”.

- Below, we see the BST and Hash Table representation.



BST

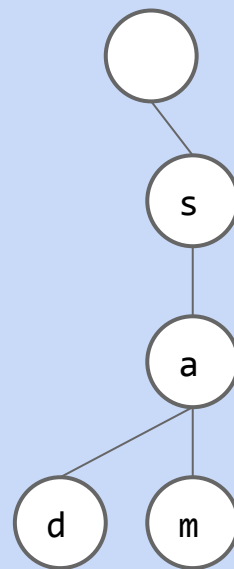


Hash Table

Tries: Each Node Stores One Character

For String keys, we can use a “Trie”. Key ideas:

- Every node stores only one letter.
- Nodes can be shared by multiple keys.

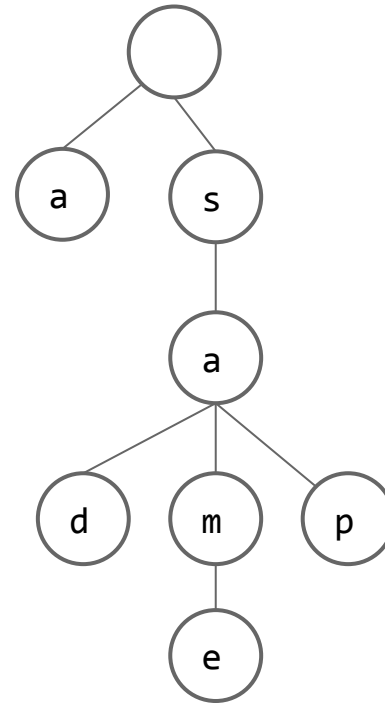


Above, we show the results of adding “sam” and “sad”. Use your intuition to try to insert the remaining items “sap”, “same”, “a”, and “awls”.

Tries: Each Node Stores One Character

For String keys, we can use a “Trie”. Key ideas:

- Every node stores only one letter.
- Nodes can be shared by multiple keys.

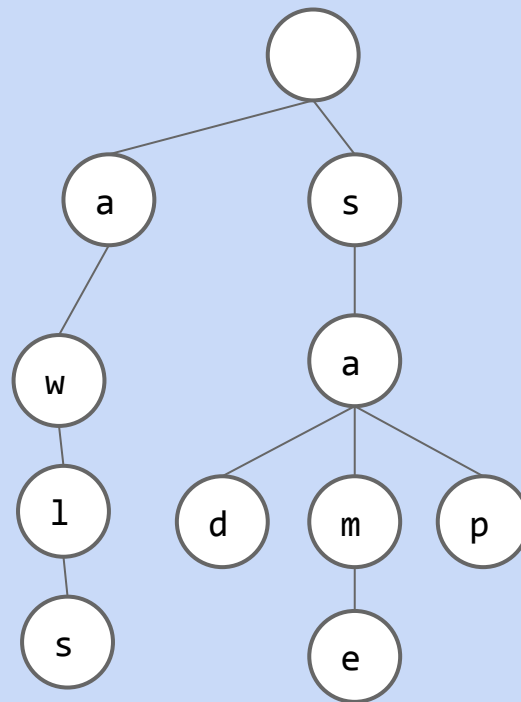


Above, we show the results of adding “sam” and “sad”. Use your intuition to try to insert the remaining items “sap”, “same”, “a”, and “awls”.

Tries: Each Node Stores One Character

For String keys, we can use a “Trie”. Key ideas:

- Every node stores only one letter.
- Nodes can be shared by multiple keys.

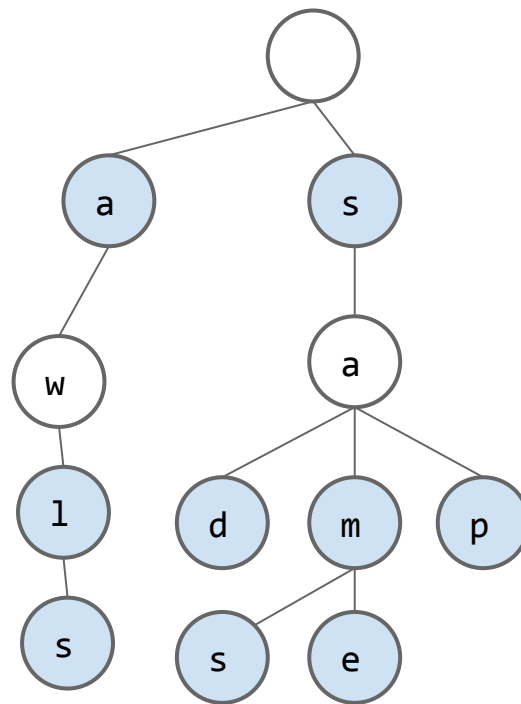


Try to figure out a way to make it clear that our set contains “sam”, “sad”, “sap”, “same”, “a”, and “awls”, but not “aw”, “awl”, “sa”, etc.

Tries: Each Node Stores One Character

For String keys, we can use a “Trie”. Key ideas:

- Every node stores only one letter.
- Nodes can be shared by multiple keys.



Try to figure out a way to make it clear that our set contains “sam”, “sad”, “sap”, “same”, “a”, and “awls”, but not “aw”, “awl”, “sa”, etc.

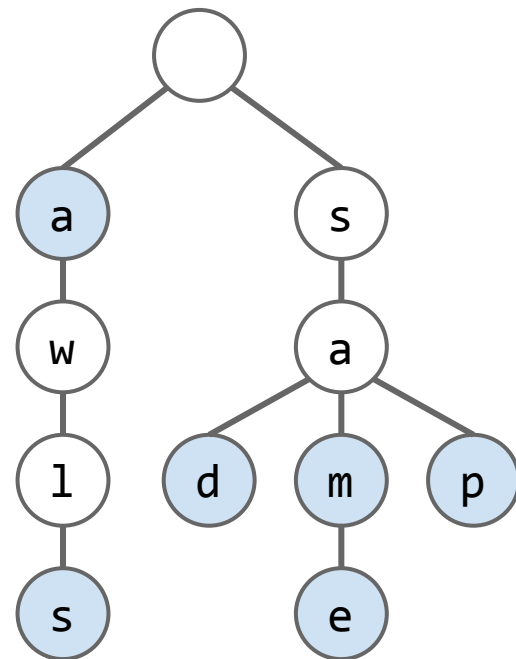
Tries: Search Hits and Misses

Suppose we insert “sam”, “sad”, “sap”, “same”, “a”, and “awls”.

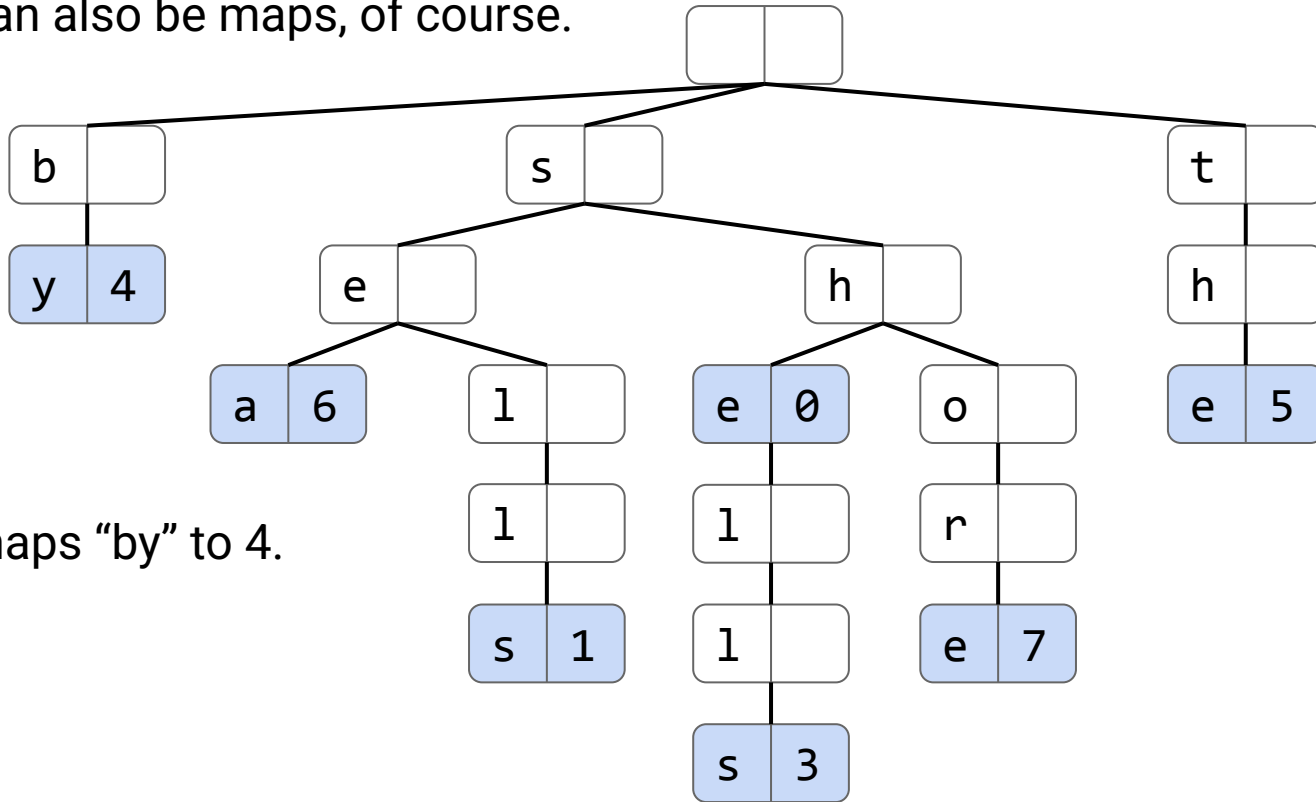
- contains(“sam”): true, blue node
 - contains(“sa”): false, white node
 - contains(“a”): true, blue node
 - contains(“saq”): false, fell off tree
- “hit”
- “miss”

Two ways to have a search “miss”:

- If the final node is white.
- If we fall off the tree, e.g. contains(“sax”).



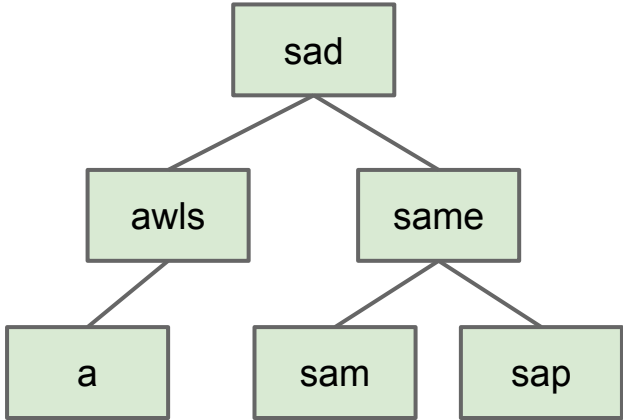
Tries can also be maps, of course.



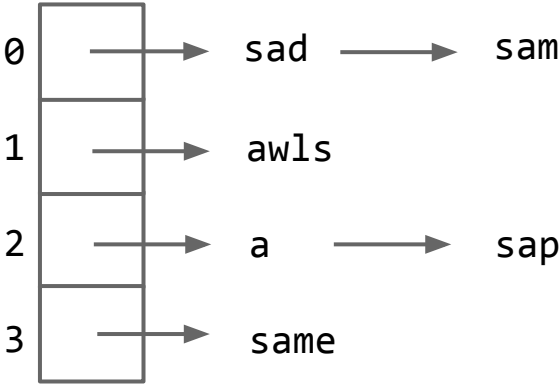
e.g. maps "by" to 4.

For an animated demo of the creation of this map, see this [demo](#) from our optional Algorithms textbook.

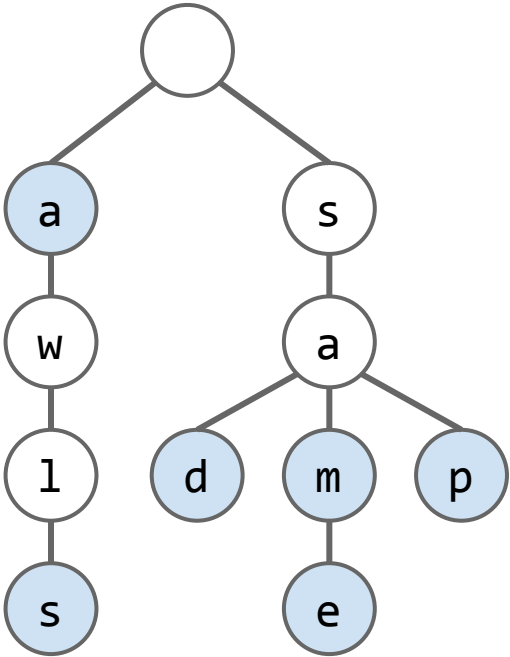
Tries: A Digit-by-Digit Set Representation



BST



HashSet



Trie

Trie:

- Short for **Retrieval** Tree.
- Inventor Edward Fredkin suggested it should be pronounced “tree”, but almost everyone pronounces it like “try”.

Why did Edward Fredkin choose that word? [[edit](#)]

Since he pronounced it homophonous to ‘tree’, didn't he realize that it was a pretty stupid choice, because that would make it impossible to distinguish the words in speech? If he was so desperate to combine ‘tree’ and ‘retrieve’, surely he could have done better? [Shinobu](#) ([talk](#)) 22:06, 5 October 2008 (UTC) [[reply](#)]

Trie Implementation and Performance

Lecture 29, CS61B, Fall 2025

Tries (Conceptual)

Trie Implementation and Performance

Alternate Child Tracking Strategies

Trie String Operations

Autocomplete

Trie Summary

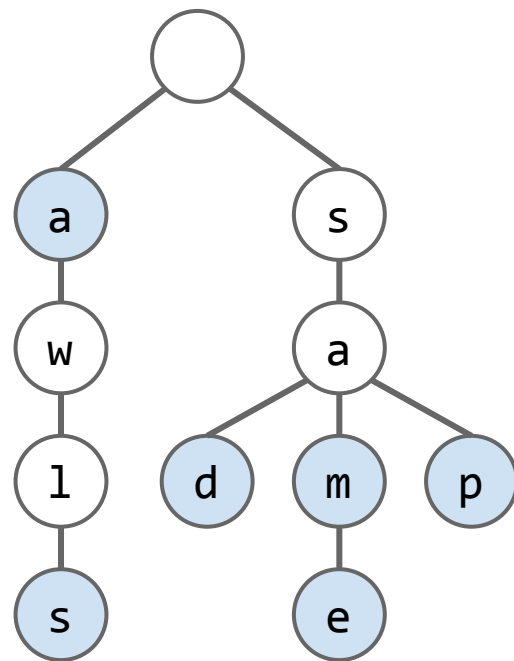
Very Basic Trie Implementation

The first approach might look something like the code below.

- Each node stores a letter, a map from c to all child nodes, and a color.

```
public class TrieSet {  
    private static final int R = 128; // ASCII  
    private Node root; // root of trie  
  
    private static class Node {  
        private char ch;  
        private boolean isKey;  
        private DataIndexedCharMap<Node> next;  
        private Node(char c, boolean b, int R) {  
            ch = c; isKey = b;  
            next = new DataIndexedCharMap<>(R);  
        }  
    }  
}
```

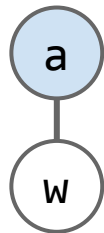
Since we know our keys are characters,
can use a DataIndexedCharMap.



Zooming in On a Node

Each DataIndexedCharMap is an array of 128 possible links, mostly null.

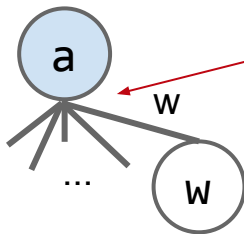
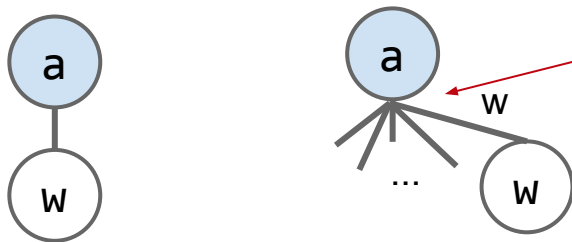
```
private static class Node {  
    private char ch;  
    private boolean isKey;  
    private DataIndexedCharMap<Node> next;  
    private Node(char c, boolean b, int R) {  
        ch = c; isKey = b;  
        next = new DataIndexedCharMap<>(R);  
    }  
}
```



Zooming in On a Node

Better drawing of a DataIndexedCharMap based trie is shown to the right.

```
private static class Node {  
    private char ch;  
    private boolean isKey;  
    private DataIndexedCharMap<Node> next;  
    private Node(char c, boolean b, int R) {  
        ch = c; isKey = b;  
        next = new DataIndexedCharMap<>(R);  
    }  
}
```



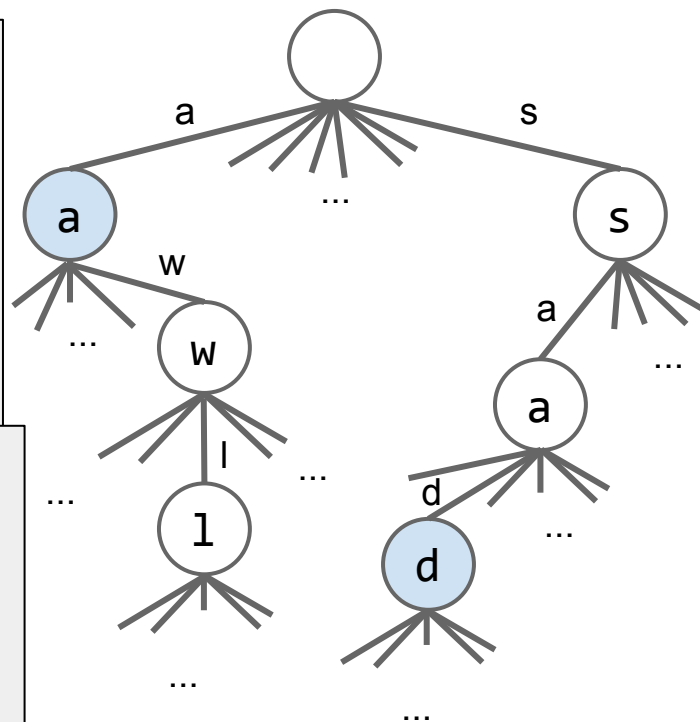
128 links, with one used, and 127 equal to null.

Very Basic Trie Implementation

If we use a `DataIndexedCharMap` to track children, every node has `R` links.

```
private static class Node {  
    private char ch;  
    private boolean isKey;  
    private DataIndexedCharMap<Node> next;  
    private Node(char c, boolean b, int R) {  
        ch = c; isKey = b;  
        next = new DataIndexedCharMap<>(R);  
    }  
}
```

```
public class DataIndexedCharMap<V> {  
    private V[] items;  
    public DataIndexedCharMap(int R) {  
        items = (V[]) new Object[R];  
    }  
    ...  
}
```

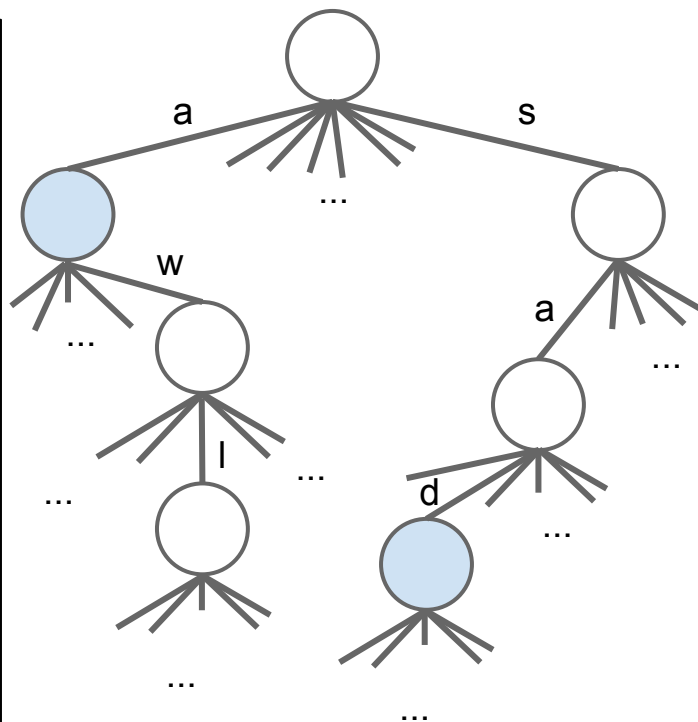


Very Basic Trie Implementation

Observation: The letter stored inside each node is actually redundant.

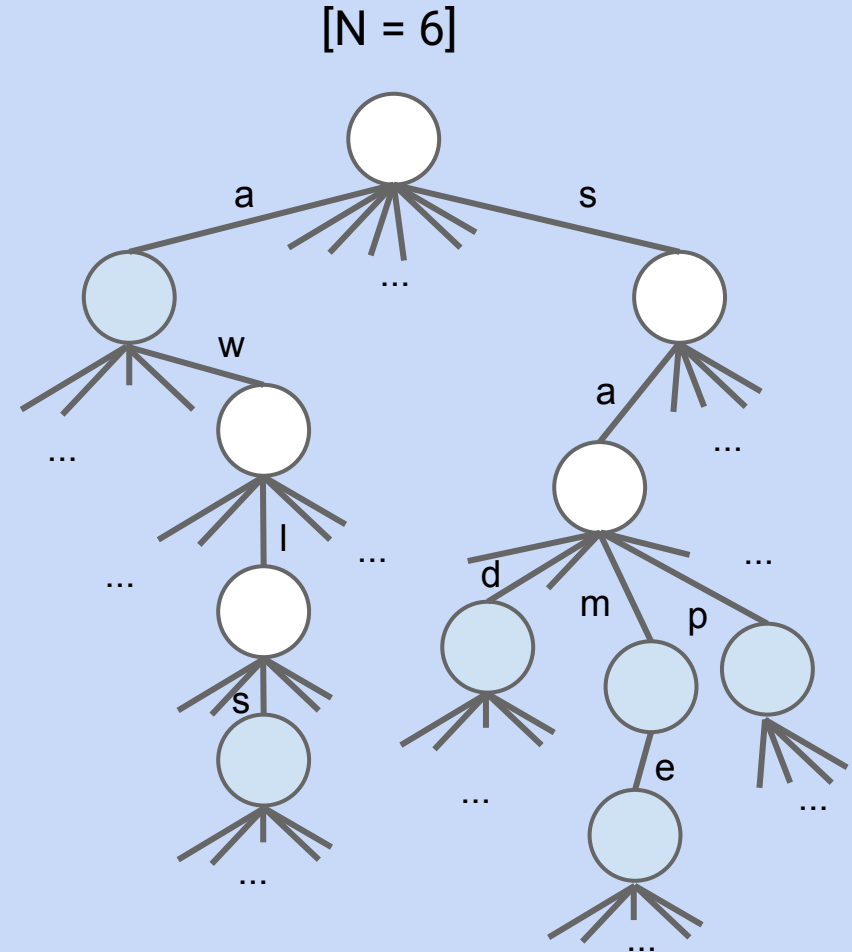
- Can remove from the representation and things will work fine.

```
public class TrieSet {  
    private static final int R = 128; // ASCII  
    private Node root; // root of trie  
  
    private static class Node {  
        private char ch;  
        private boolean isKey;  
        private DataIndexedCharMap<Node> next;  
        private Node(char c, boolean b, int R) {  
            ch = c; isKey = b;  
            next = new DataIndexedCharMap<>(R);  
        }  
    }  
}
```



Given a Trie with N keys. What is the:

- Add runtime?
- Contains runtime?



Trie Performance in Terms of N

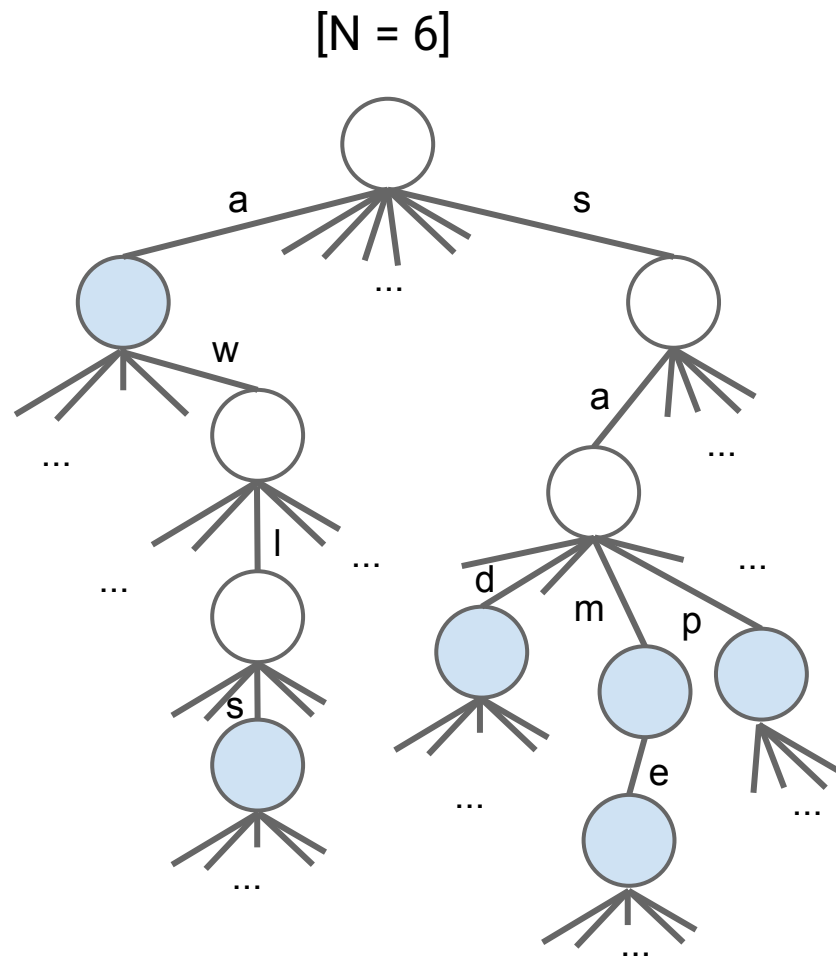
Given a Trie with N keys. What is the:

- Add runtime? $\Theta(1)$
- Contains runtime? $\Theta(1)$

Runtimes independent of number of keys!

Or in terms of L, the length of the key:

- Add: $\Theta(L)$
- Contains: $\Theta(L)$



Trie Performance in Terms of N

When our keys are strings, Tries give us slightly better performance on contains and add.

Runtimes treating length of keys as a constant

	key type	get(x)	add(x)
Balanced BST	comparable	$\Theta(\log N)$	$\Theta(\log N)$
Resizing Separate Chaining Hash Table	hashable	$\Theta(1)$ assuming even spread	$\Theta(1)$ on average, assuming even spread
data indexed array	chars	$\Theta(1)$	$\Theta(1)$
Tries	Strings	$\Theta(1)$	$\Theta(1)$

One downside of the DataIndexedCharMap-based Trie is the huge memory cost of storing R links per node.

- Wasteful because most links are unused in real world usage.

Alternate Child Tracking Strategies

Lecture 29, CS61B, Fall 2025

Tries (Conceptual)

Trie Implementation and Performance

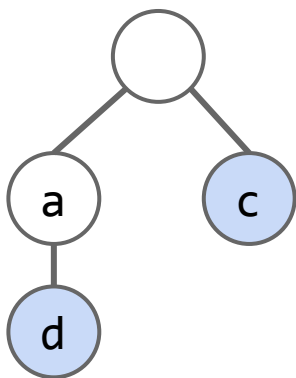
Alternate Child Tracking Strategies

Trie String Operations

Autocomplete

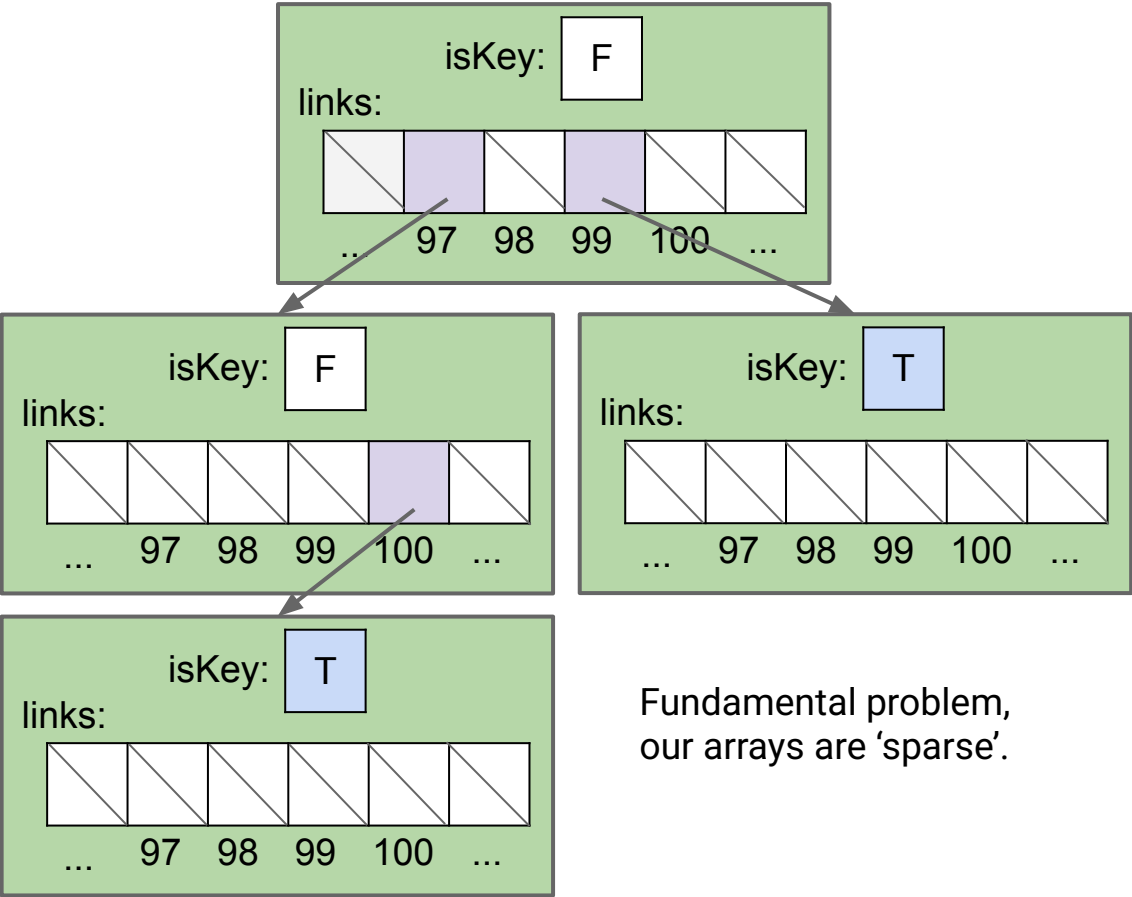
Trie Summary

The DataIndexedCharMap Trie

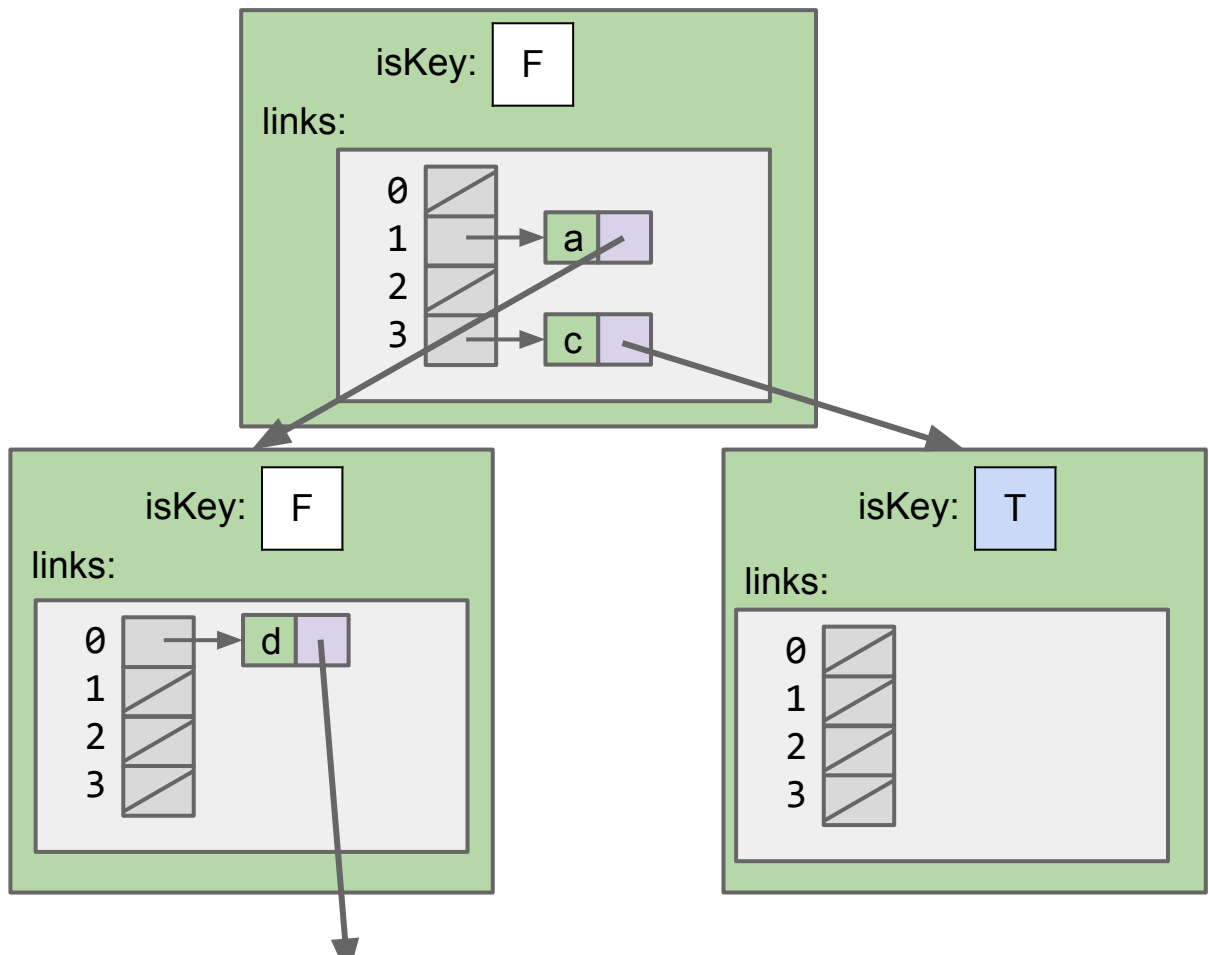
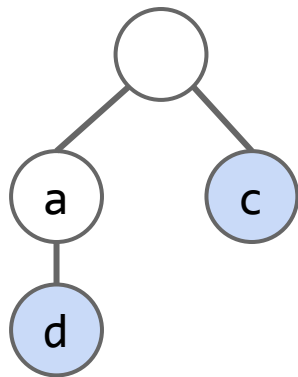


Can use ANY kind of map from character to node, e.g.

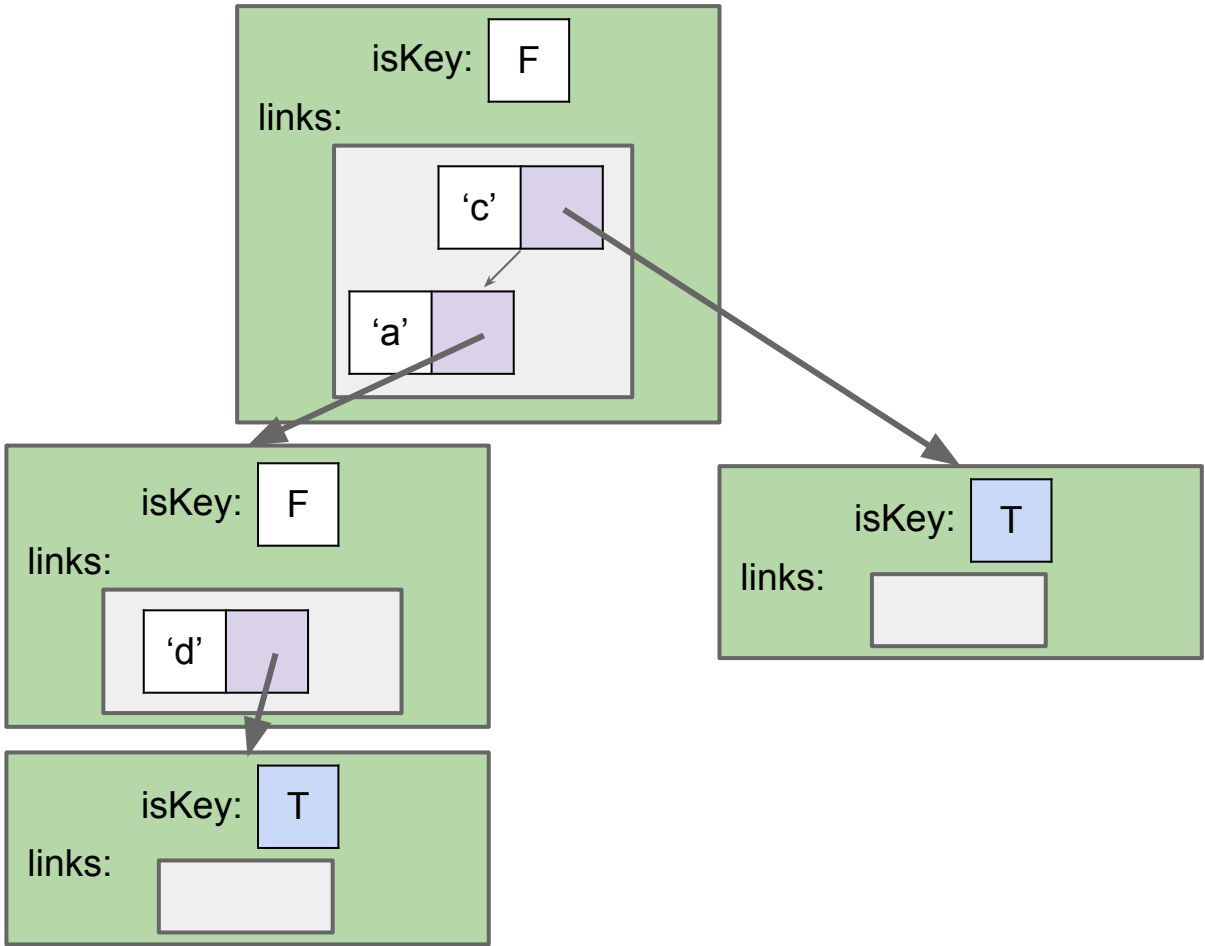
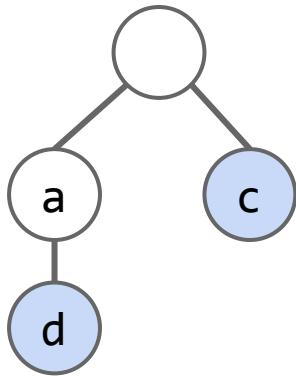
- BST
- Hash Table



Alternate Idea #1: The Hash-Table Based Trie



Alternate Idea #2: The BST-Based Trie

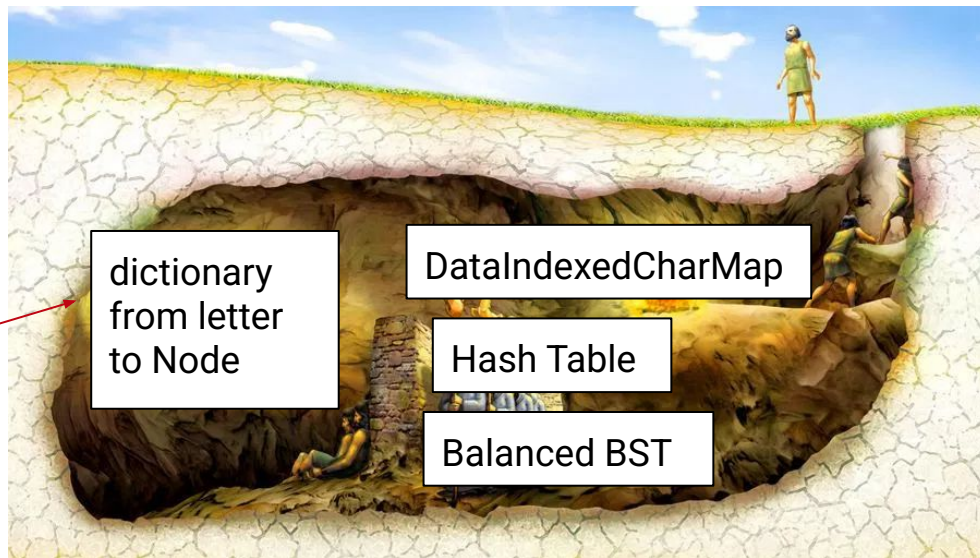


The Three Trie Implementations

When we implement a Trie, we have to pick a map to our children

- DataIndexedCharMap: Very fast, but memory hungry.
- Hash Table: Almost as fast, uses less memory.
- Balanced BST: A little slower than Hash Table, uses similar amount of memory?

this.next



Performance of the DataIndexedCharMap, BST, and Hash Table Trie

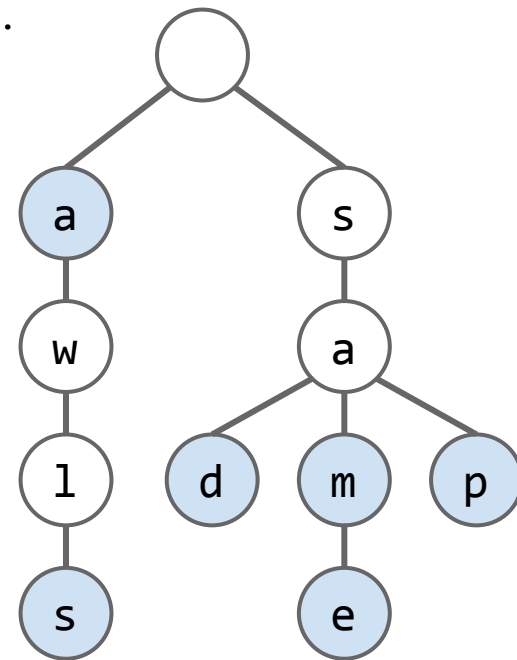
Using a BST or a Hash Table to store links to children will usually use less memory.

- DataIndexedCharMap: 128 links per node.
- BST: C links per node, where C is the number of children.
- Hash Table: C links per node.
- Note: Cost per link is higher in BST and Hash Table.

Using a BST or a Hash Table will take slightly more time.

- DataIndexedCharMap is $\Theta(1)$.
- BST is $O(\log R)$, where R is size of alphabet.
- Hash Table is $O(R)$, where R is size of alphabet.

Since R is fixed (e.g. 128), can think of all 3 as $\Theta(1)$.



Trie Performance in Terms of N

When our keys are strings, Tries give us slightly better performance on contains and add.

- Using BST or Hash Table will be slightly slower, but more memory efficient.
- Would have to do computational experiments to see which is best for your application.

Runtimes treating length of keys as a constant

	key type	get(x)	add(x)
Balanced BST	comparable	$\Theta(\log N)$	$\Theta(\log N)$
Resizing Separate Chaining Hash Table	hashable	$\Theta(1)$ assuming even spread	$\Theta(1)$ on average, assuming even spread
data indexed array	chars	$\Theta(1)$	$\Theta(1)$
Tries (BST, Hash Table, Data Indexed Char Map)	Strings	$\Theta(1)$	$\Theta(1)$

... but where Tries really shine is their efficiency with special string operations!

Trie String Operations

Lecture 29, CS61B, Fall 2025

Tries (Conceptual)

Trie Implementation and
Performance

Alternate Child Tracking Strategies

Trie String Operations

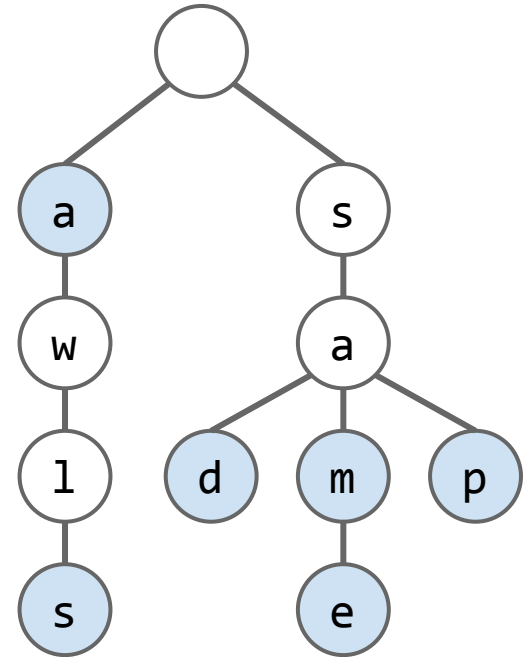
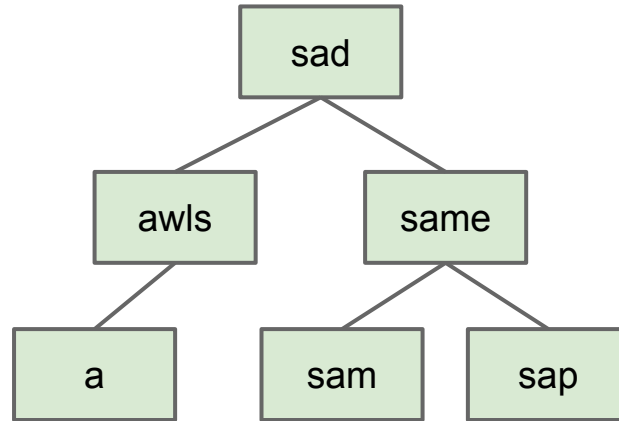
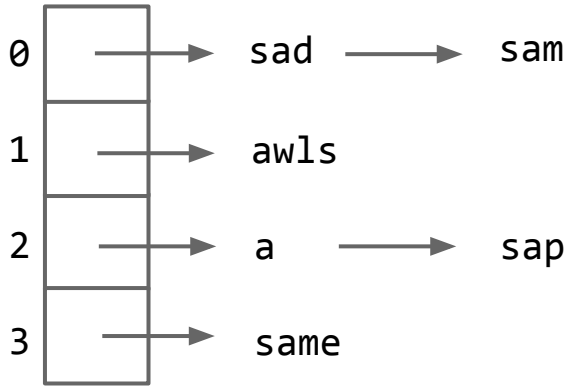
Autocomplete

Trie Summary

String Specific Operations

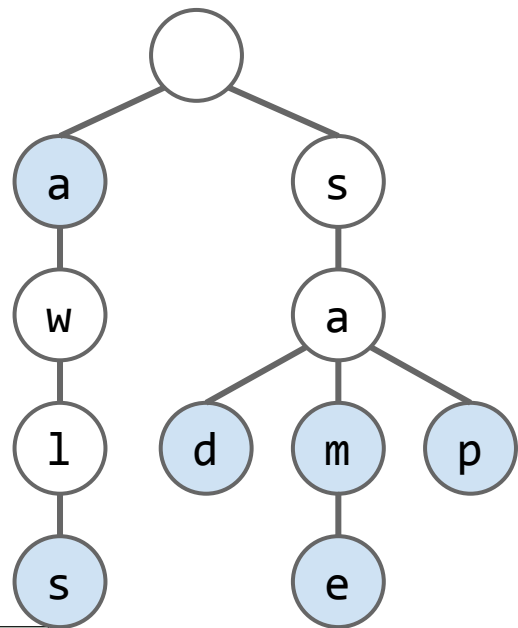
Theoretical asymptotic speed improvement is nice. But the **main appeal of tries** is their ability to efficiently support string specific operations like **prefix matching**.

- Finding all keys that match a given prefix: `keysWithPrefix("sa")`
- Finding longest prefix of: `longestPrefixOf("sample")`



Prefix Matching Operations

Theoretical asymptotic speed improvement is nice. But the main appeal of tries is their ability to efficiently support string specific operations like prefix matching.



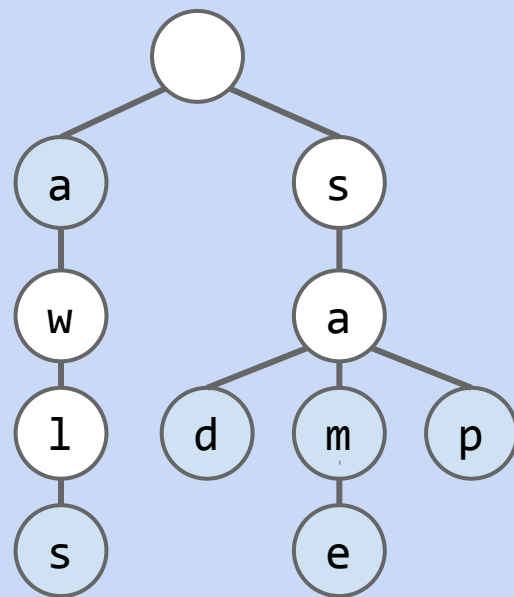
Examples:

- Finding the longest prefix of a string:
`longestPrefixOf("sample")`
 - Result: sam
- Finding all keys that match a given prefix:
`keysWithPrefix("sa")`
 - Result: [sad, sam, same, sap]

Challenging Warmup Exercise: Collecting Trie Keys

Challenging Exercise: Give an algorithm for collecting all the keys in a Trie.

`collect()` returns ["a", "awls", "sad", "sam", "same", "sap"]



Challenging Warmup Exercise: Collecting Trie Keys

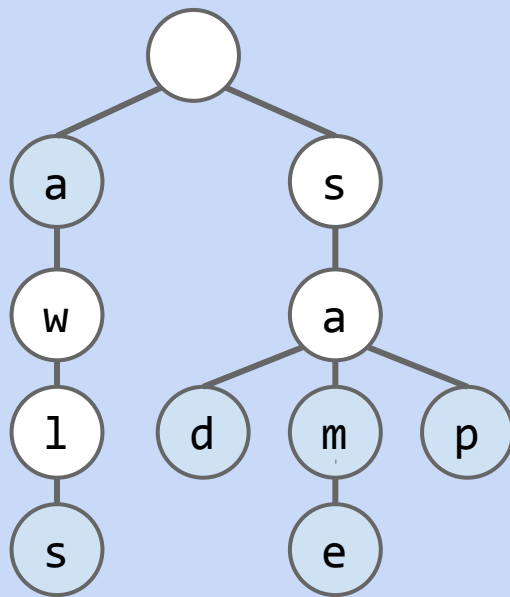
Challenging Exercise: Give an algorithm for collecting all the keys in a Trie.

`collect()` returns ["a", "awls", "sad", "sam", "same", "sap"]

`collect()`:

- Create an empty list of results `x`.
- For character `c` in `root.next.keys()`:
 - Call `colHelp(c, x, root.next.get(c))`.
- Return `x`.

`colHelp(String s, List<String> x, Node n):`



Challenging Warmup Exercise: Collecting Trie Keys

Challenging Exercise: Give an algorithm for collecting all the keys in a Trie.

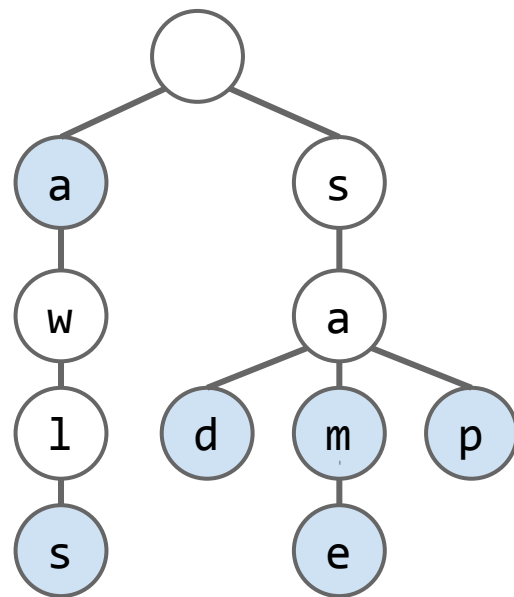
`collect()` returns ["a", "awls", "sad", "sam", "same", "sap"]

`collect()`:

- Create an empty list of results `x`.
- For character `c` in `root.next.keys()`:
 - Call `colHelp(c, x, root.next.get(c))`.
- Return `x`.

`colHelp(String s, List<String> x, Node n):`

- If `n.isKey`, then `x.add(s)`.
- For character `c` in `n.next.keys()`:
 - Call `colHelp(s + c, x, n.next.get(c))`



Challenging Warmup Exercise: Collecting Trie Keys

Challenging Exercise: Give an algorithm for collecting all the keys in a Trie.

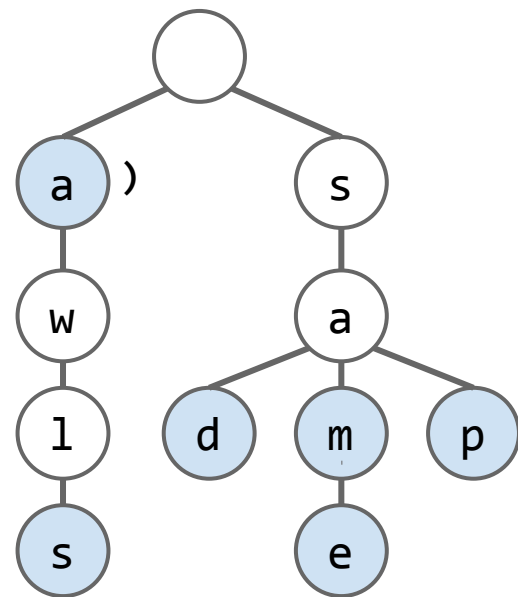
`collect()`:

- Create an empty list of results `x`.
- For character `c` in `root.next.keys()`:
 ○ Call `colHelp(c, x, root.next.get(c))`.

`x = []`
- Return `x`.

`colHelp(String s, List<String> x, Node n):` `colHelp("a", x, a)`

- If `n.isKey`, then `x.add(s)`.
- For character `c` in `n.next.keys()`:
 ○ Call `colHelp(s + c, x, n.next.get(c))`



Challenging Warmup Exercise: Collecting Trie Keys

Challenging Exercise: Give an algorithm for collecting all the keys in a Trie.

`collect()`:

- Create an empty list of results `x`.
- For character `c` in `root.next.keys()`:
 - Call `colHelp(c, x, root.next.get(c))`.
- Return `x`.

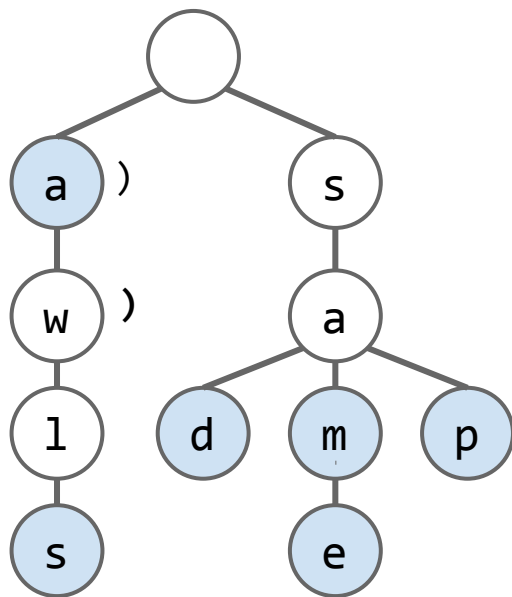
`x = ["a"]`

`colHelp(String s, List<String> x, Node n):`

- If `n.isKey`, then `x.add(s)`.
- For character `c` in `n.next.keys()`:
 - Call `colHelp(s + c, x, n.next.get(c))`

`colHelp("a", x, a)`

`colHelp("aw", x, w)`



Challenging Warmup Exercise: Collecting Trie Keys

Challenging Exercise: Give an algorithm for collecting all the keys in a Trie.

`collect()`:

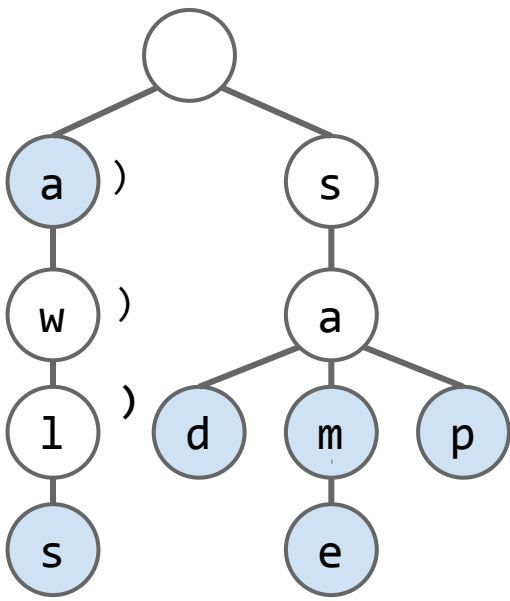
- Create an empty list of results `x`.
- For character `c` in `root.next.keys()`:
 - Call `colHelp(c, x, root.next.get(c))`.
- Return `x`.

`x = ["a"]`

`colHelp(String s, List<String> x, Node n):`

- If `n.isKey`, then `x.add(s)`.
- For character `c` in `n.next.keys()`:
 - Call `colHelp(s + c, x, n.next.get(c))`

`colHelp("a", x,`
`colHelp("aw", x,`
`colHelp("awl", x,`



Challenging Warmup Exercise: Collecting Trie Keys

Challenging Exercise: Give an algorithm for collecting all the keys in a Trie.

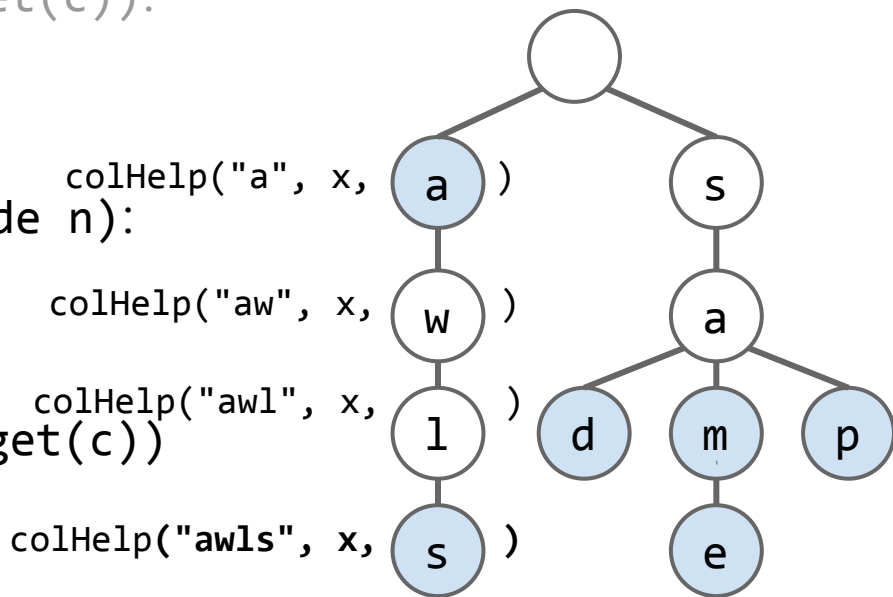
`collect()`:

- Create an empty list of results `x`.
- For character `c` in `root.next.keys()`:
 - Call `colHelp(c, x, root.next.get(c))`.
- Return `x`.

`x = ["a"]`

`colHelp(String s, List<String> x, Node n):`

- If `n.isKey`, then `x.add(s)`.
- For character `c` in `n.next.keys()`:
 - Call `colHelp(s + c, x, n.next.get(c))`



Challenging Warmup Exercise: Collecting Trie Keys

Challenging Exercise: Give an algorithm for collecting all the keys in a Trie.

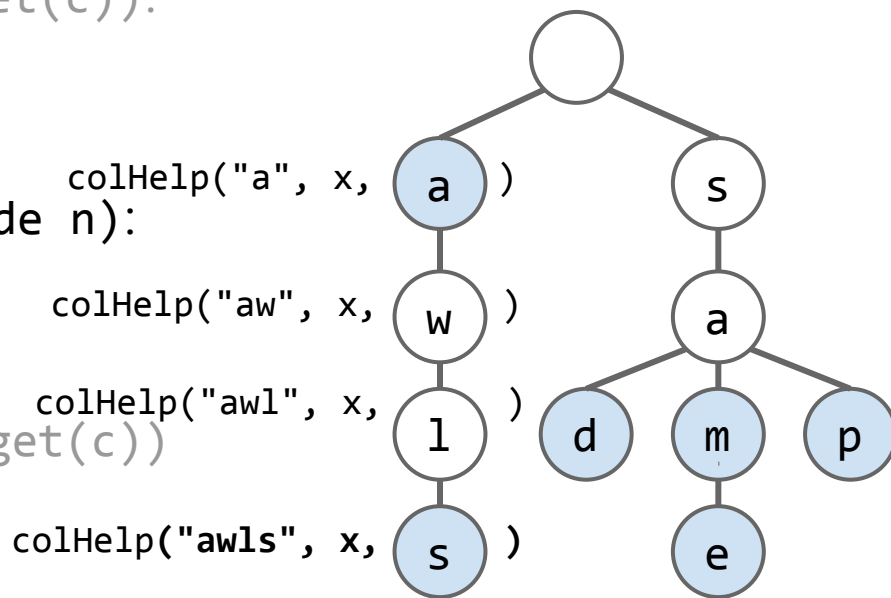
`collect()`:

- Create an empty list of results `x`.
- For character `c` in `root.next.keys()`:
 - Call `colHelp(c, x, root.next.get(c))`.
- Return `x`.

`x = ["a", "awls"]`

`colHelp(String s, List<String> x, Node n):`

- If `n.isKey`, then `x.add(s)`.
- For character `c` in `n.next.keys()`:
 - Call `colHelp(s + c, x, n.next.get(c))`



Challenging Warmup Exercise: Collecting Trie Keys

Challenging Exercise: Give an algorithm for collecting all the keys in a Trie.

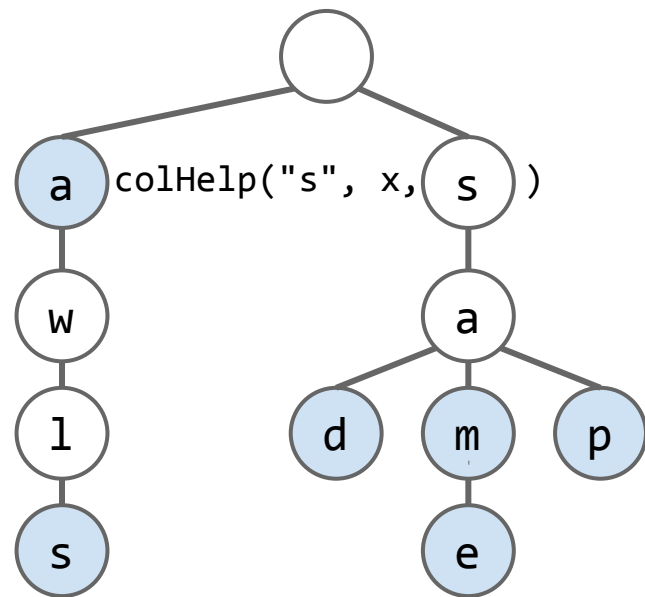
`collect()`:

- Create an empty list of results `x`.
- For character `c` in `root.next.keys()`:
 - Call `colHelp(c, x, root.next.get(c))`.
- Return `x`.

`colHelp(String s, List<String> x, Node n):`

- If `n.isKey`, then `x.add(s)`.
- For character `c` in `n.next.keys()`:
 - Call `colHelp(s + c, x, n.next.get(c))`

`x = ["a", "awls"]`



Challenging Warmup Exercise: Collecting Trie Keys

Challenging Exercise: Give an algorithm for collecting all the keys in a Trie.

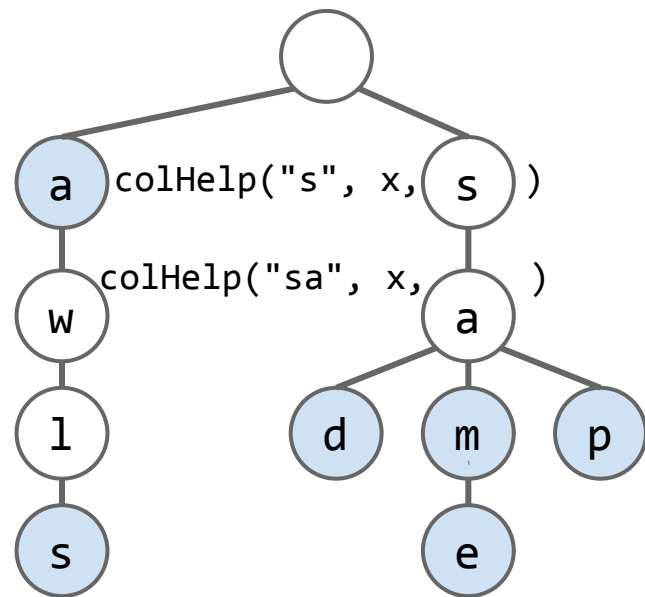
`collect()`:

- Create an empty list of results `x`.
- For character `c` in `root.next.keys()`:
 - Call `colHelp(c, x, root.next.get(c))`.
- Return `x`.

`colHelp(String s, List<String> x, Node n):`

- If `n.isKey`, then `x.add(s)`.
- For character `c` in `n.next.keys()`:
 - Call `colHelp(s + c, x, n.next.get(c))`

`x = ["a", "awls"]`



Challenging Warmup Exercise: Collecting Trie Keys

Challenging Exercise: Give an algorithm for collecting all the keys in a Trie.

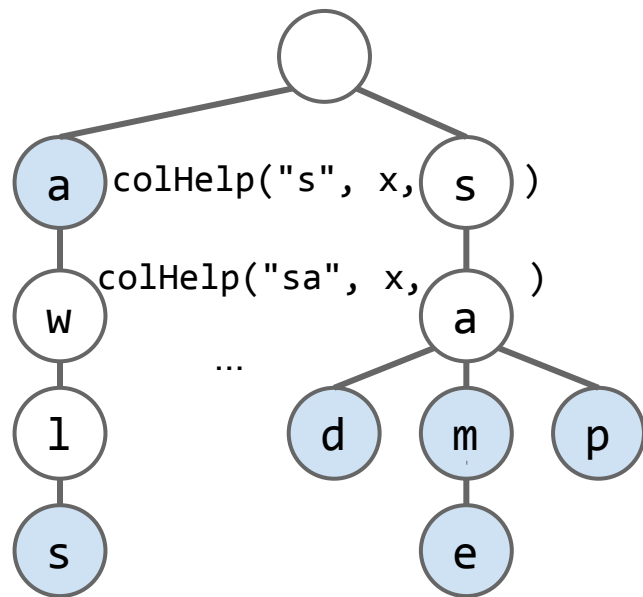
`collect()`:

- Create an empty list of results `x`.
- For character `c` in `root.next.keys()`:
 - Call `colHelp(c, x, root.next.get(c))`.
- Return `x`.

`colHelp(String s, List<String> x, Node n):`

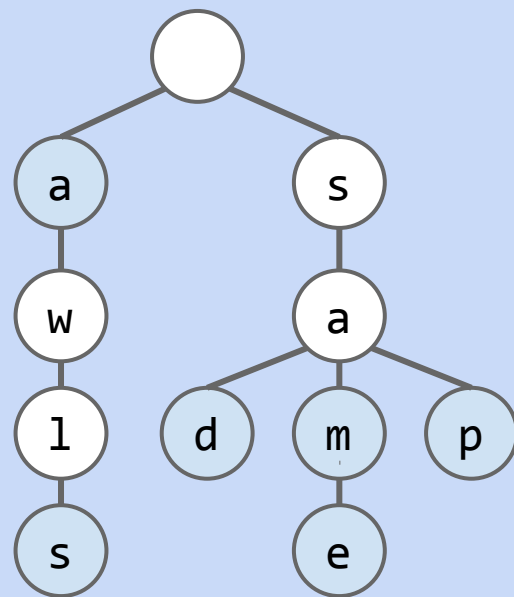
- If `n.isKey`, then `x.add(s)`.
- For character `c` in `n.next.keys()`:
 - Call `colHelp(s + c, x, n.next.get(c))`

`x = ["a", "awls", "sad",
"sam", "same", "sap"]`



Challenge: Give an algorithm for `keysWithPrefix`.

- Example: `keysWithPrefix("sa")` is `["sad", "sam", "sap"]`.



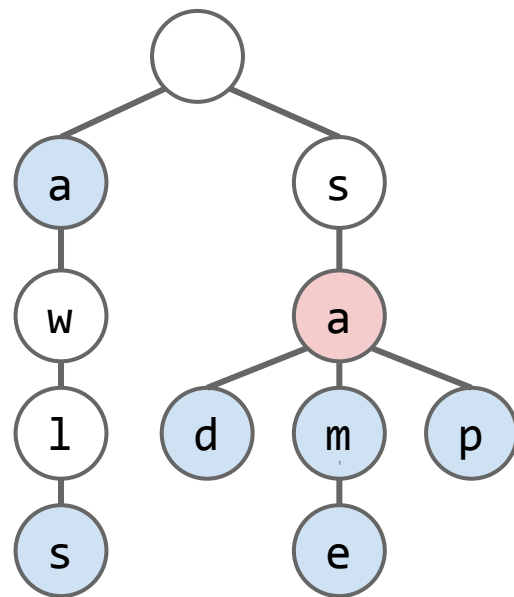
Challenge: Give an algorithm for `keysWithPrefix`.

- Example: `keysWithPrefix("sa")` is `["sad", "sam", "same", "sap"]`.

Algorithm:

- Find the node α corresponding to the string (in pink).
- Create an empty list `x`.
- For character `c` in `$\alpha$ .next.keys()`:
 - Call `colHelp("sa" + c, x,  $\alpha$ .next.get(c))`.

Another common operation: `LongestPrefixOf`. See lab.



Optional, see videos if you're curious.

Autocomplete

Lecture 29, CS61B, Fall 2025

Tries (Conceptual)

Trie Implementation and
Performance

Alternate Child Tracking Strategies

Trie String Operations

Autocomplete

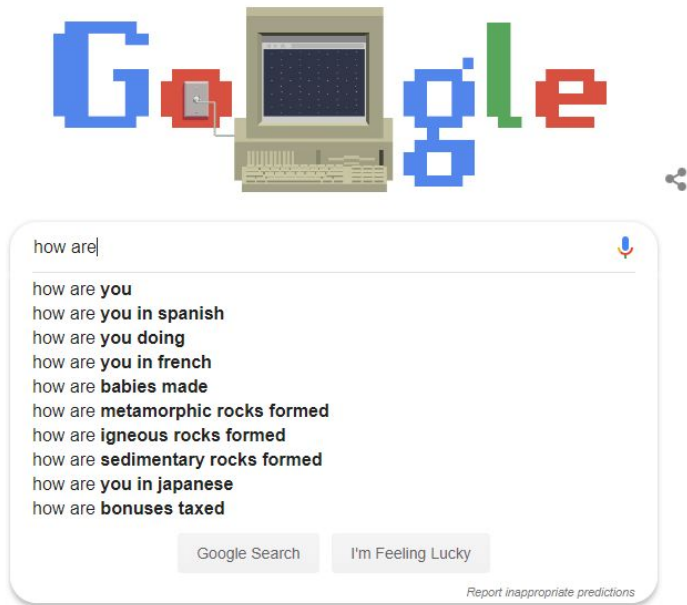
Trie Summary

The Autocomplete Problem

Example, when I type “how are” into Google, I get 10 results, shown to the right.

One way to do this is to create a Trie based map from strings to values

- Value represents how important Google thinks that string is.
- Can store billions of strings efficiently since they share nodes.
- When a user types in a string “hello”, we:
 - Call `keyswithPrefix("hello")`.
 - Return the 10 strings with the highest value.



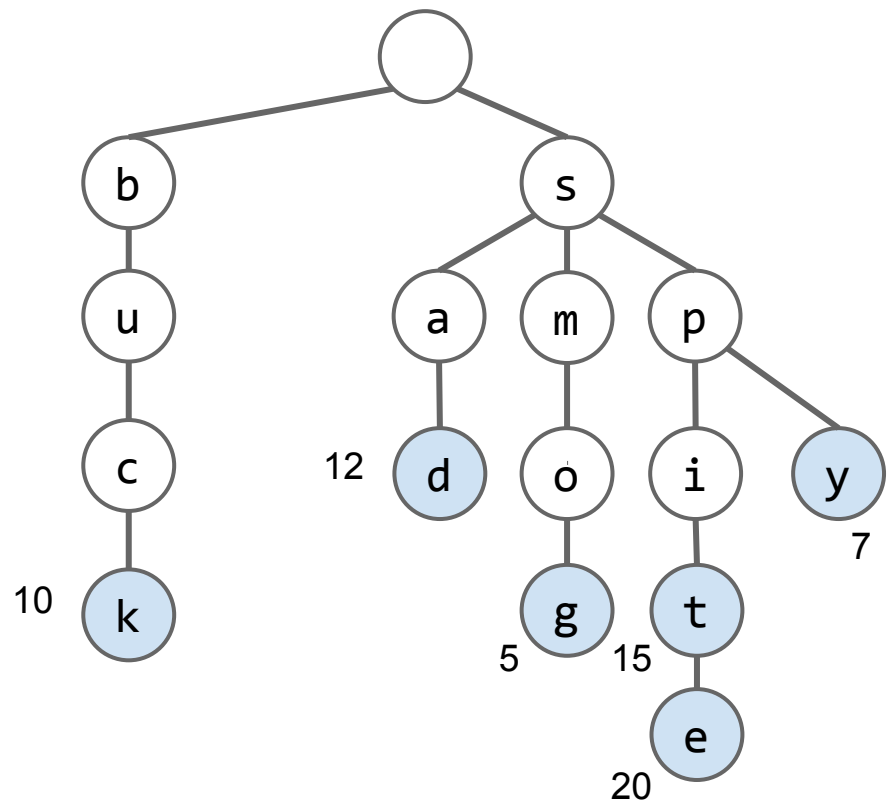
Autocomplete Example, for Top Three Matches

Suppose we have six strings with values shown below:

- buck: 10
- sad: 12
- smog: 5
- spit: 15
- spite: 20
- spy: 7

If the user types “s”, we:

- Call `keysWithPrefix("s")`.
 - sad, smog, spit, spite, spy
- Return the three keys with highest value.
 - spit, spite, sad



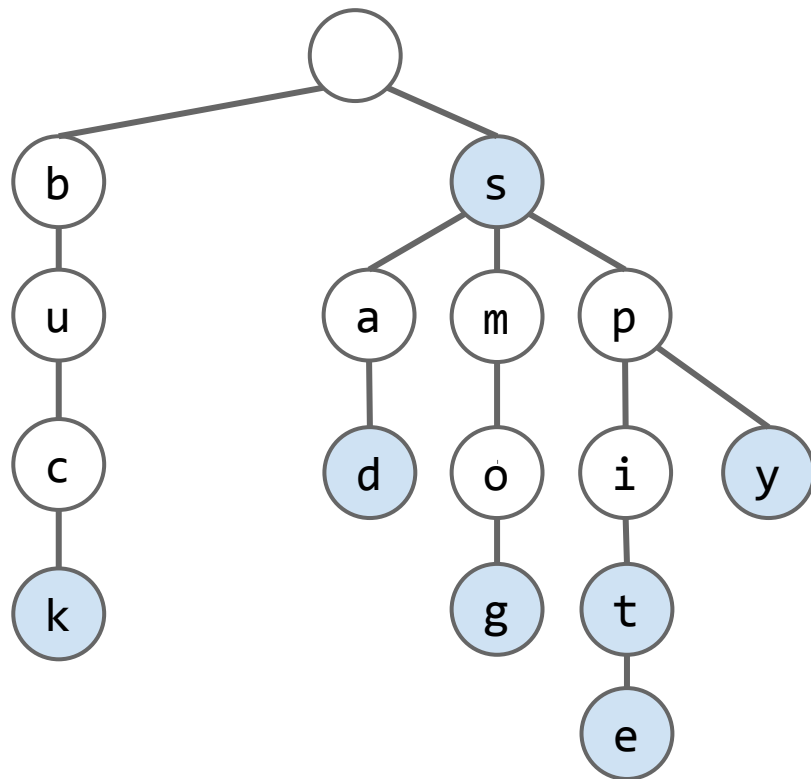
Autocomplete Example, for Top Three Matches

Suppose we have six strings with values shown below:

- buck: 10
- sad: 12
- smog: 5
- spit: 15
- spite: 20
- spy: 7

If the user types "s", we:

- Call `keysWithPrefix("s")`.
 - sad, smog, spit, spite, spy
- Return the three keys with highest value.
 - spit, spite, sad



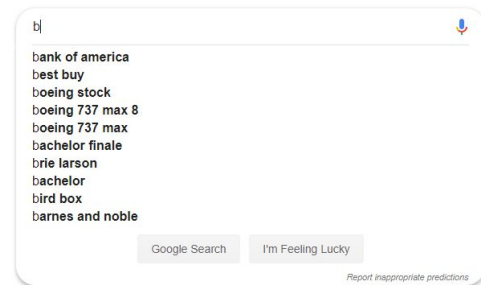
The Autocomplete Problem

One way to do this is to create a Trie based Dictionary that maps strings to values.

- When a user types in a string hello, we:
 - Call `keysWithPrefix("hello")`.
 - Return the ten strings with the highest value.

The approach above has one major flaw. If we enter a short string, the number of keys with the appropriate prefix will be too big.

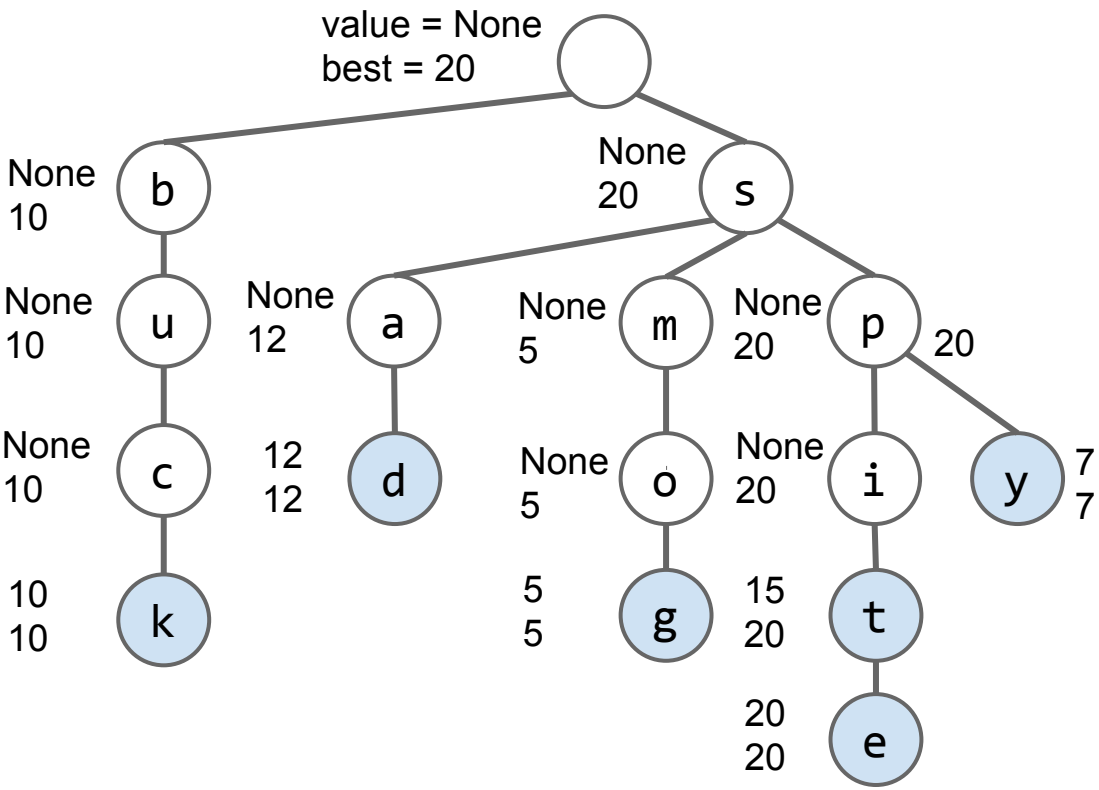
- We are collecting billions of results only to keep 10!
- This is extremely inefficient.



A More Efficient Autocomplete

One way to address this issue:

- Each node stores its own value, as well as the value of its best substring.



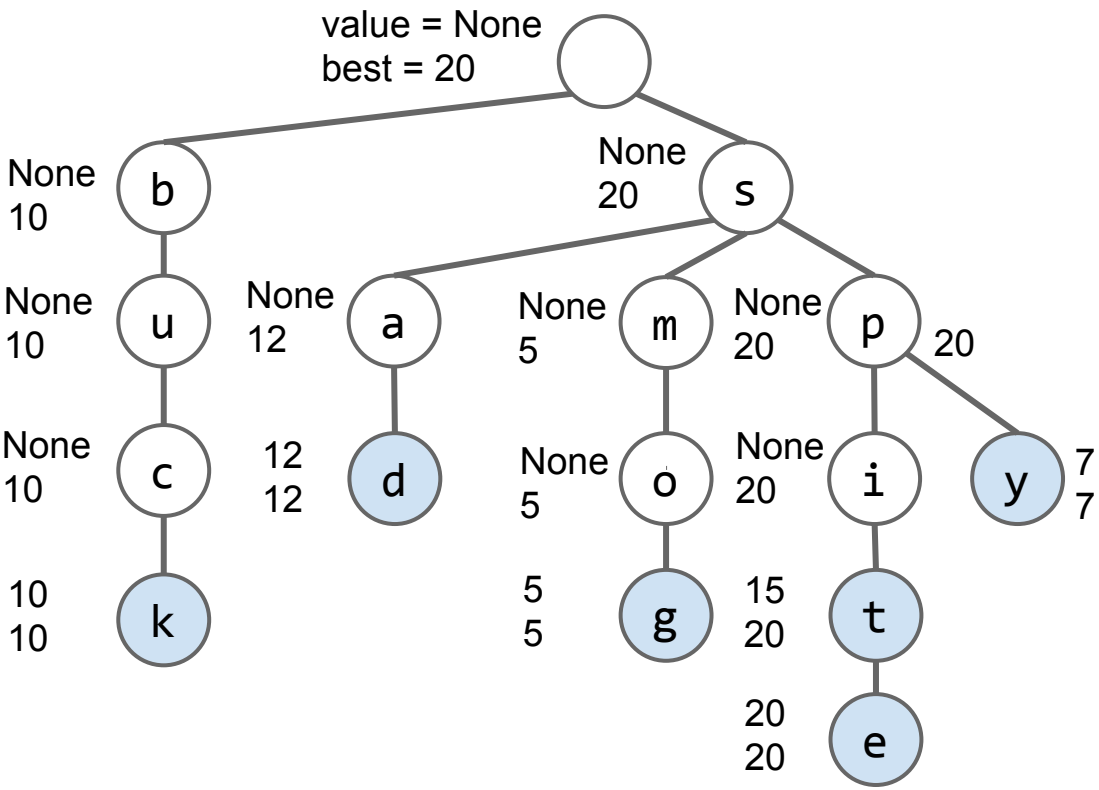
A More Efficient Autocomplete

One way to address this issue:

- Each node stores its own value, as well as the value of its best substring.

Search will consider nodes in order of “best”.

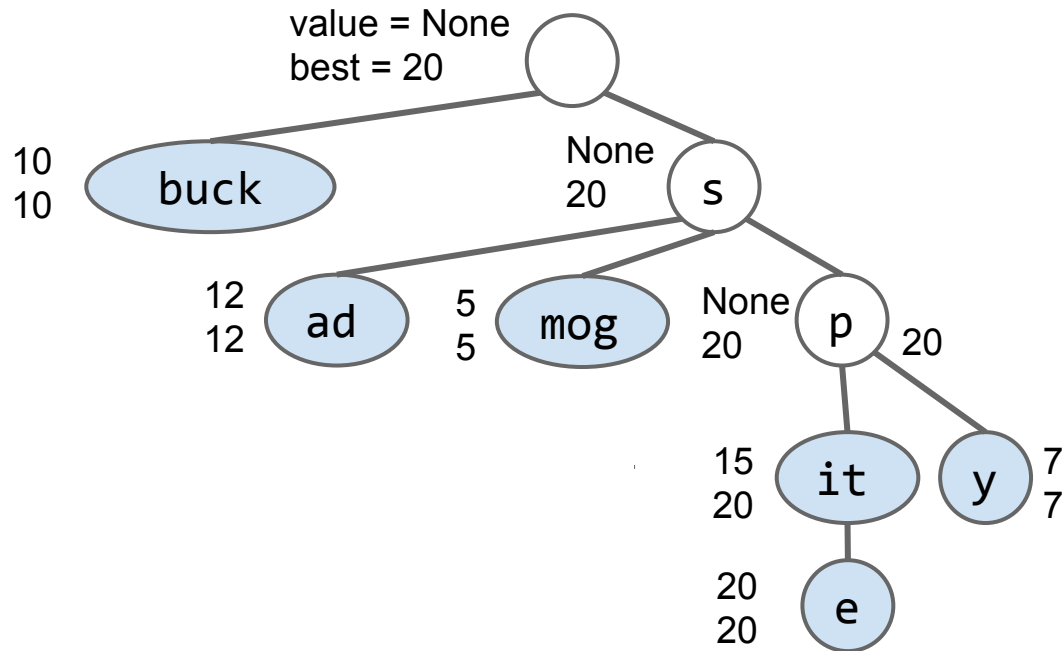
- Consider ‘sp’ before ‘sm’.
- Can stop when top 3 matches are all better than best remaining.



Even More Efficient Autocomplete

Can also merge nodes that are redundant!

- This version of trie is known as a “radix tree” or “radix trie”.
- Won’t discuss.



Trie Summary

Lecture 29, CS61B, Fall 2025

Tries (Conceptual)

Trie Implementation and
Performance

Alternate Child Tracking Strategies

Trie String Operations

Autocomplete

Trie Summary

When your key is a string, you can use a Trie:

- Theoretically better performance than hash table or search tree.
- Have to decide on a mapping from letter to node. Three natural choices:
 - DataIndexedCharMap, i.e. an array of all possible child links.
 - Bushy BST.
 - Hash Table.
- All three choices are fine, though hash table is probably the most natural.
- Supports special string operations like `longestPrefixOf` and `keysWithPrefix`.
 - `keysWithPrefix` is the heart of important technology like autocomplete.
 - Optimal implementation of Autocomplete involves use of a priority queue!

Bottom line: Data structures interact in beautiful and important ways!

More generally, we can sometimes take special advantage of our key type to improve our sets and maps.

- Example: Tries handle String keys. Allow for fast string specific operations.
- Note: There are many other types of string sets/maps out there.
 - Suffix Trees ([Link](#)).
 - DAWG ([Link](#)).
 - Won't discuss in our course.

